Blind (Successfully?) Leading the Blind: Collaborative LLM Reasoning in Lean Theorem Proving

Stanford CS229 Project

Henry Wang
Department of Computer Science
Stanford University
henwang@stanford.edu

Tom Liu
Department of Computer Science
Stanford University
liumaizi@stanford.edu

Abstract

Formal theorem proving requires proofs that are both mathematically correct and accepted by a proof assistant, usually in Lean. While large language models (LLMs) have shown promise in reasoning, their performance on Lean theorem proving remains limited. We investigate whether LLM-informed inference-time search can improve Lean theorem proving without additional training. Using Gemini 2.5 Flash, we compare a multi-attempt verifier-feedback baseline against a Tree of Thoughts approach on 126 LeanDojo v2 Mathlib problems. The baseline achieved 22.22% accuracy after five attempts, while Tree of Thoughts solved 37.30% of problems by maintaining multiple candidate proof paths and performing tactic-level search guided by Lean verification and LLM evaluation. These results suggest that structured search can substantially improve performance without fine-tuning.

1 Introduction

Formal theorem proving offers a way to verify mathematical arguments with machine-checkable rigor. In systems such as Lean, a theorem is not considered proven because it is convincing to a human reader; instead, every step must be accepted by a formal verifier.

This makes theorem proving a promising area for machine learning: if language models can generate valid Lean tactics (ideas that advance a proof one step at a time), they may help automate parts of mathematical reasoning, software verification, and formalized mathematics. However, theorem proving is also difficult for current language models because a proof must be not only plausible in natural language, but also syntactically valid, type-correct, and accepted by the Lean kernel.

Current models have poor performance on Lean theorem-proving tasks, particularly on proofs that require multi-step reasoning and precise theorem selection. In this project, we investigate if search-based prompting and LLM evaluation can improve LLM performance on Lean theorem proving.

Our method is partially inspired by Adam Brown's Google DeepMind's talk at Stanford's Future of Mathematics Symposium. He presented on how language models perform better when they generate intermediate reasoning before producing a solution. Tree of Thoughts extends this idea by exploring multiple reasoning paths and selecting the most promising candidates. This is a natural fit for theorem proving because a Lean proof is built step by step: each tactic changes the proof state, and a bad early tactic can make the rest of the proof difficult or impossible. Maintaining several candidate paths may help recover the model from poor local choices (He also wished us "best of luck").

We compare two approaches using Gemini 2.5 Flash. The first is a multi-attempt verifier-feedback baseline, where the model repeatedly generates complete proofs and receives Lean error feedback after failures. The second is a Tree of Thoughts approach that explores multiple candidate tactic paths and uses verification and evaluation to guide search. We study whether structured inference-time search improves theorem-proving performance over repeated retries alone.

The input to our algorithms is a JSON file containing the string of a Lean theorem statement together with a string of the initial Lean tactic state (imagine being given a math problem on an exam, the initial tactic state is simply what is written down).

Both models outputs a string of a complete Lean tactic script, which is then checked using Lean via Pantograph. Pantograph allows automated Lean verification checks via Python. A problem is counted as solved only if the generated tactics are accepted by Lean.

2 Related Work

Recent work has shown that language models can be useful for formal theorem proving. GPT-f used transformer language models to generate proofs in Metamath, showing that language models could guide automated theorem proving (Polu and Sutskever, 2020). PACT extended this direction to Lean by training on proof artifacts extracted from Lean's kernel-level proof terms, improving theorem-proving performance on held-out Lean problems (Han et al., 2021). LeanDojo provides an open-source environment for extracting Lean data, interacting with proof states, and evaluating theorem-proving models; it also introduced ReProver, a retrieval-augmented prover for Mathlib (Yang et al., 2023). Our project uses the LeanDojo v2 setting, but instead of training a new model or adding retrieval, we focus on inference-time proof generation.

Prompting and search for language model reasoning has shown to boost performance, similar to Adam Brown's talk. Chain-of-Thought prompting showed that language models often perform better when they generate intermediate reasoning steps before producing an answer (Wei et al., 2022). Tree of Thoughts extends this idea by exploring multiple possible reasoning paths and choosing among them, which is useful for tasks requiring planning or backtracking (Yao et al., 2023). This is a natural fit for theorem proving because a Lean proof is built step by step: each tactic changes the proof state, and a bad early tactic can make the rest of the proof difficult or impossible.

Finally, Graph of Thoughts generalizes tree-based reasoning by allowing thoughts to be combined, refined, or revisited in a graph structure (Besta et al., 2023). We do not implement Graph of Thoughts, but it motivates future work because different proof branches may discover different useful ideas that could work together. Overall, prior work suggests that language models can generate useful proof steps, but they benefit from external verification, feedback, and search.

3 Dataset and Features

We evaluated our methods on a fixed set of 126 theorem-proving problems randomly sampled from the LeanDojo v2 Mathlib dataset (Yang et al., 2023). Each problem comes from a theorem in Mathlib, the main mathematical library for Lean. Originally, we wanted to evaluate on 140 theorems however our Gemini API credits ran out so Tree of Thoughts only ran on 126 problems. Both the baseline and Tree of Thoughts method were evaluated on the same fixed set.

We filtered the dataset to examples that could be initialized successfully in our Lean (v4.29.1) and Pantograph (v0.3.15) setup via "Mathlib.Tactic," a lean module import that introduces a lot of proof tactics like rings, etc. This helped avoid counting environment or parsing failures as model failures.

For each theorem, the dataset includes the theorem name, the theorem statement, and the initial Lean tactic state. One example from the dataset is shown below:


```
{"name": "Int.isUnit_eq_one_or",
 "goal": "\u2200 (u v : \u2124) (hu : IsUnit u), u = 1 \u2228 u = -1",,
 "initial_state": "u v : \u2124\nhu : IsUnit u\n\u22a2 u = 1 \u2228 u = -1",
 "givens": ["u v : \u2124", "hu : IsUnit u"],
 "target": "u = 1 \u2228 u = -1"}
```


In this example, the problem defines u to be an integer and v its multiplicative inverse. The problem claims if they are both integers, then u is either 1 or -1

It illustrates the format of our task: the model is not asked to write a natural-language proof, but to produce Lean tactics that transform the current tactic state into a completed proof.

4 Methods

We have two algorithms: a multi-attempt verifier-feedback baseline and the "Tree of Thoughts"

4.1 Multi-Attempt Verifier-Feedback Baseline

Our baseline directly prompts the LLM to generate a complete Lean tactic proof from the theorem statement and the initial tactic state. After the model generates a proof, we run the tactic script through Lean using Pantograph. If Lean accepts the proof and returns no remaining goals, the theorem is marked as solved. If the proof fails, we store the failed tactic script along with the Lean/Pantograph error message. The model is then prompted again with the original theorem statement, the original tactic state, all previous failed attempts, and the corresponding verifier feedback. Importantly, each retry is asked to generate a complete replacement proof from the original proof state, rather than continuing from the failed intermediate state.

We allow the baseline up to five attempts per theorem. This makes the baseline stronger than simple "one-shot" proof generation because it can use verifier feedback to correct syntax errors, invalid tactics, or incomplete proofs. However, it still searches in a limited way: each attempt is a full proof script, and the model does not explicitly maintain multiple partial proof paths.

4.2 Tree of Thoughts

Our model is adapted from the "Tree of Thoughts" model. Traditionally, LLMs commit to a single chain of thought. This is dangerous as if it makes a mistake early on, everything afterward is affected. This is especially relevant in mathematical proofs because proving something often explores multiple intermediate proof strategies. In "Tree of Thoughts," the LLM generates multiple possible thoughts and evaluates how promising each thought is. It then keeps the best branches and continues to expand on those thoughts.

Our model utilizes Gemini-2.5-Flash API calls as both "Prover" agent and an "Evaluator" agent. The workflow for each problem goes as follows:

1. The search space and initial proof state (goals needed to be proven) is initialized
2. Prover proposes three proposal tactics as a JSON file
3. Evaluator runs a deterministic Lean check through PantoGraph to see if proposal is valid or solves the problem
4. Evaluator evaluates each tactic and gives it short feedback and score via a JSON file
5. The three most promising tactics, selected based on Lean validity and LLM evaluation scores, are chosen for further exploration
6. Prover is provided with up to five previous attempts, Lean and LLM feedback on previous five attempts
7. Prover proposes another three new tactics for each new tactic
8. Steps 3 to 7 are repeated 10 times, at which the model gives up and fails to solve the problem.

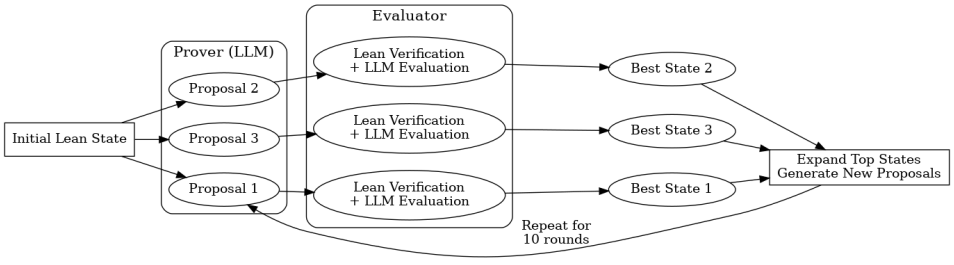


Figure 1: Tree of Thoughts proof workflow

Importantly, the prover isn't asked to solve the problem in one-step, rather generate intermediate tactics that eventually lead to final proof state.

5 Experiments / Results / Discussion

We do not have traditional hyperparameters like learning rate because we weren't training a traditional neural network, we were just using fixed pretrained LLM-based prover on a held-out benchmark of Lean goals. Instead, our hyperparameters controlled the proof-search procedure.

Specifically, the prover generates three candidate tactics. The evaluator then checks each tactic with Lean/Pantograph and retains the top three valid proof states. We run the ten rounds of iterations.

We chose these values as a compromise between exploration and cost: increasing the number of candidates or rounds gives the search more chances to recover from bad tactics but also increases LLM calls. We did not tune these hyperparameters extensively because API cost and runtime were limiting factors; instead, we selected small values that allowed evaluation over the full dataset.

Our primary metrics is proof success rate, i.e. amount of proofs it solved.

$$\text{Accuracy} = \frac{\text{Number of solved theorems}}{\text{Total number of theorem problems}}$$

We also track number of rounds each problems takes to see if the LLM is simply "one-shotting" the proof or taking advantage of search paths given by Tree of Thoughts or feedback loop in baseline.

We also measure error rate in Tree of Thoughts because the LLM Prover and Evaluator are expected to return JSON parseable string containing proposed tactics, feedback, and scores. However, since LLM outputs are stochastic, the model sometimes returns malformed JSON. For example, starting the file with """ ''' json {..." when only one """" is expected. When this happens, the entire search breaks.

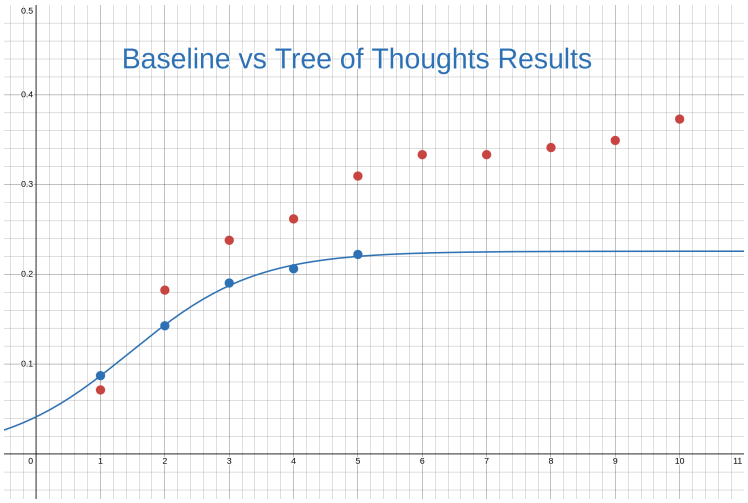


Figure 2: Baseline model accuracy as the number of allowed attempts increases. The dashed red line shows the Tree of Thoughts accuracy, while the blue curve shows the multi-attempt baseline.

Quantitatively, the baseline solved 11/126 problems after one attempt, 18/126 after two attempts, 24/126 after three attempts, 26/126 after four attempts, and 28/126 after five attempts. This corresponds to a final accuracy of 22.22%. The curve increases quickly but begins to flatten after three attempts. To visualize this saturation effect, we fit a logistic curve to the baseline results. The fitted curve had an R^2 value of 0.9974 and appeared to converge below 23%, suggesting that giving the model more retry attempts with verifier feedback would likely provide only limited additional gains.

In contrast, the Tree of Thoughts method solved 47/126 (37.30%) problems, 39/126 when only limited to five attempts (30.95%). This is substantially higher than the apparent baseline saturation point. However, Tree of Thoughts also had errors on 14/126 problems due to unparseable JSON text.

Qualitatively, Tree of Thoughts performed best on goals solvable by common Lean tactics such as "intro", "simp", "rfl", "ring", or "inferinstance". It struggled on goals requiring specific Mathlib lemma names, where the model often hallucinated theorem names or used plausible but invalid tactic syntax.

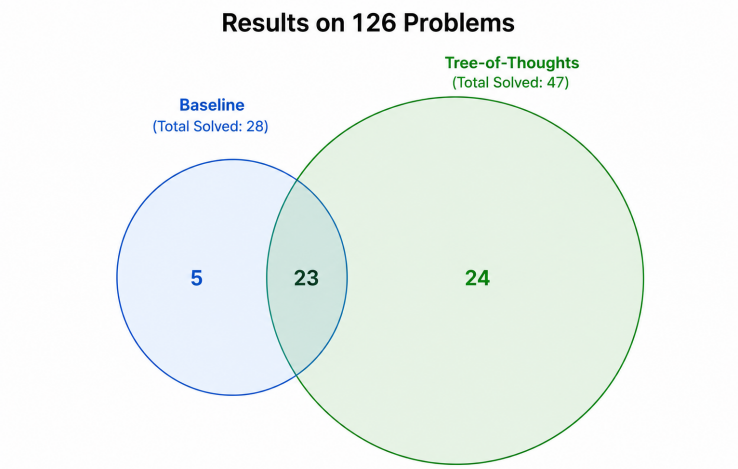


Figure 3: Tree of Thoughts recovered 23 of the 28 baseline successes (82.1%) and contributed 24 additional unique solutions, suggesting that it largely subsumes the baseline.

A major failure point was LLM outputting nonexistent theorem names: "exact Subsingleton.punit". This shows the model understands the rough idea but lacks reliable Mathlib name recall.

At the same time, model also overrelies on these common Lean tactics. These worked well for some problems, but failed when the goal required a specific lemma or when "simp" made no progress.

6 Future Work

If we had more time and more credits, we first would have also extended our baseline to 10 rounds of feedback to match our Tree of Thoughts model. Furthermore, we would also like to finetune our Prover model to the newest Lean documentation. We found the main error in the network is how the LLM may hallucinate Lean statements despite having the right idea. This may be explained by how Lean v4.29.1 was released April 14th, 2026 whereas Gemini 2.5 Flash was released June 17th, 2025.

We also hope to observe collaborativeness between different LLMs. For example, have one OpenAI model, one Meta model, etc. Specifically, instead of one model proposing multiple solutions, each model proposes their own solution plus each model independently evaluating each other's proposal. This approach may reduce correlated errors and hallucinations, since models trained on different datasets and architectures often make different mistakes. Furthermore, independent cross-evaluation can provide a more stronger assessment of proof quality than self-evaluation alone, potentially leading to stronger candidate selection and improved overall search performance.

7 Conclusion

In this project, we investigated whether inference-time search can improve Lean theorem proving. Tree of Thoughts solved 47/126 problems (37.30%), substantially outperforming the baseline's LLM 11/126 (8.73%) accuracy trying to directly generating the proof and even 28/126 (22.22%) accuracy after five attempts. The results suggest that Tree of Thoughts' gains are not simply due to using more tokens, retries, or compute. While the multi-attempt baseline improved with feedback, its performance quickly plateaued, whereas Tree of Thoughts continued improving to 37.3% accuracy. It indicates that structured search is more effective than repeated retries alone. By maintaining multiple candidate proof states and making tactic-level decisions, Tree of Thoughts can recover from early mistakes that would otherwise prevent a proof from succeeding.

8 Contributions

- **Tom Liu**: Worked on literature review regarding LLMs performance in mathematics, gathering LeanDojo datasets, running baseline model, getting API keys from Gemini and OpenRouter, setting up API calls as helper functions.
- **Henry Wang**: Worked on establishing Tree of Thoughts environment, creating Prover and Evaluator framework, finetuning prompts for Prover and Evaluator agents, filtering dataset to work with Prover and Evaluator agents, setting up Lean and Pantograph Environment.

References

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2023. Graph of thoughts: Solving elaborate problems with large language models. *arXiv preprint arXiv:2308.09687*.

Jesse Michael Han, Jason Rute, Yuhuai Wu, Edward Ayers, and Stanislas Polu. 2021. Proof artifact co-training for theorem proving with language models. *arXiv preprint arXiv:2102.06203*.

Stanislas Polu and Ilya Sutskever. 2020. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*.

Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. 2023. Leandojo: Theorem proving with retrieval-augmented language models. *arXiv preprint arXiv:2306.15626*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*.

A Prompts

A.1 Prover Prompt

```
You are a Lean theorem-proving agent.

Return exactly a JSON array of up to {numThoughts} objects. Each object must have:
- "tactic": one Lean tactic line
- "rationale": short reason
- "confidence": number from 0 to 1

Do not include markdown.

Theorem:
{state["problem"]}

Lean state so far:
{state.get("lean_state", "") or "(empty)"}

Previous failed attempts:
{state.get("failed_candidates", [])}

Lessons from previous failed attempts:
{state.get("lessons", [])}
```

```
Hard rules:
- Do not repeat failed tactics.
- Do not reuse identifiers that Lean reported as unknown.
- Do not reuse syntax styles that caused parse errors.
- If a lemma name was unknown, try a different lemma name or a tactic that avoids naming it.
```

A.2 Evaluator Prompt

```
You are evaluating a Lean tactic attempt.

Lean environment:
- Lean 4 project
- Mathlib version: leanprover-community/mathlib v4.29.1
- Backend: Pantograph ‘goal_tactic‘


Theorem:
{checked_state["problem"]}

Candidate tactic:
{checked_state["candidate"]}

Lean accepted tactic:
{checked_state["valid"]}

Solved:
{checked_state["solved"]}

Lean feedback:
{checked_state["feedback"]}

Current Lean state after tactic:
{checked_state["lean_state"]}

Return exactly JSON:
{{
"feedback": "short critique",
"score": number from 0 to 1
}}

Do not include markdown.
```

B Code Availability

Source code, prompts, and experiment scripts are available at `https://github.com/pianomaster99/Fermats2ndLastTheorem`.